

evaluation

Useful utilities for model evaluations.

The Java ds-api counterpart is `com.feedzai.commons.dsapi.Evaluation` .

ABSOLUTE_IMPORTANCE module-attribute

```
ABSOLUTE_IMPORTANCE = 'Absolute Importance'
```

FEATURE module-attribute

```
FEATURE = 'Feature'
```

IMPORTANCE module-attribute

```
IMPORTANCE = ['#Recall', '[$Recall]']
```

RELATIVE_IMPORTANCE module-attribute

```
RELATIVE_IMPORTANCE = 'Relative Importance'
```

ConfusionMatrix

```
ConfusionMatrix(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents a Confusion Matrix returned by Evaluation objects.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

accuracy

```
accuracy() -> float
```

Accuracy metric.

Returns:

Type	Description
<code>float</code>	Accuracy value.

alert_rate

```
alert_rate() -> float
```

Alert Rate metric (a.k.a., positive predictive rate, or positive rate).

Returns:

Type	Description
float	Alert Rate value if denominator is > 0 else 0.

call_metric [¶](#)

```
call_metric(metric: Metric) -> float
```

Hides the metric dictionary from the caller to preserve its mutability when calling a metric.

Parameters:

Name	Type	Description	Default
metric ¶	Metric	Desired metric to use.	<i>required</i>

Returns:

Type	Description
float	Metric value.

false_discovery_rate [¶](#)

```
false_discovery_rate() -> float
```

False Discovery Rate metric.

Returns:

Type	Description
float	False Discovery Rate value if denominator is > 0 else 0.

false_negative_rate [¶](#)

```
false_negative_rate() -> float
```

False Negative Rate metric.

Returns:

Type	Description
float	False Negative Rate value if denominator is > 0 else 0.

negative_predictive_value [¶](#)

```
negative_predictive_value() -> float
```

Negative Predictive Value metric.

Returns:

Type	Description
float	Negative Predictive Value value if denominator is > 0 else 0.

positive_predictive_value [¶](#)

```
positive_predictive_value() -> float
```

Positive Predictive Value metric.

Returns:

Type	Description
float	Positive Predictive Value value if denominator is > 0 else 0.

specificity [¶](#)

```
specificity() -> float
```

Specificity metric.

Returns:

Type	Description
float	Specificity value if denominator is > 0 else 0.

Evaluation [¶](#)

```
Evaluation(pyjnius_java)
```

Bases: [JavaWrapper](#)

Represents an Evaluation in Pulse.

subclasses [class-attribute](#) [instance-attribute](#) [¶](#)

```
subclasses: List[JavaWrapper] = []
```

get_feature_importance_results [¶](#)

```
get_feature_importance_results(target_value: str | bool | int) -> DataFrame
```

Extract the results of the absolute and relative feature importance evaluated in terms of #Recall and \$Recall. Feature importance based on \$Recall is only available if a cost field has been defined in the model evaluation config.

Parameters:

Name	Type	Description	Default
target_value ¶	(str, bool, int or float)	The target value for which the feature importance results will be extracted.	<i>required</i>

Returns:

Type	Description
DataFrame	Pandas df with feature importance results, ordered by descending values of the #Recall-based feature importance.

Examples:

```
>>> model_eval.get_feature_importance_results('true')
```

get_result [↗](#)

```
get_result(target_value: bool | int | str = True, cost_field: str | None = None) -> list
```

Return the evaluation result.

Parameters:

Name	Type	Description	Default
target_value ↗	(bool, int, str)	Value for the target positive class.	True
cost_field ↗	str	Name (desc) of the cost field. By using this parameter, the result will be a confusion matrix with cost-related metrics.	None

Returns:

Type	Description
list	Sequence of confusion matrices, optionally with cost metrics.

get_score_field [↗](#)

```
get_score_field(target_value: bool | int | str = True) -> str
```

Get the score field for a given target value.

Raises:

Type	Description
ValueError	When the target value is not found in the list of possible fields.

get_score_field_names [↗](#)

```
get_score_field_names() -> dict[str, str]
```

Return the possible name of scored fields in this evaluation.

Examples:

```
>>> evaluation.get_score_field_names()
```

Returns:

Name	Type	Description
fields	tuple of str	The names of the fields associated with scores in this evaluation.

Name	Type	Description
values	tuple of str	The possible values from the target label (sanitized).

run_feature_importance [¶](#)

```
run_feature_importance(desired_fpr: float = 0.01, sample_rate: float = 0.3, random_seed: int | None = None, job_profile: JobProfile | None = None, job_properties: dict[str, str] | None = None, wait: bool = False) -> None
```

Run feature importance on the model evaluation object.

The values of the features are shuffled, and the loss in (transaction or cost) Recall with respect to the true Recall is computed, for each feature individually.

Parameters:

Name	Type	Description	Default
desired_fpr ¶	float	The desired FPR for feature importance. Defaults to 0.01.	0.01
sample_rate ¶	float	Sampling rate to be applied prior to feature importance. Defaults to 0.3.	0.3
random_seed ¶	int	Random seed, for reproducibility purposes. Defaults to a random integer between 1 and 1000000.	None
job_profile ¶	JobProfile	Pulse UI custom job profile.	None
job_properties ¶	Mapping of str	Map of spark configurations for the job. Defaults to Pulse's Properties Profile.	None
wait ¶	bool	Waits or not for feature importance to end. Defaults to False.	False

Examples:

```
>>> model_eval.run_feature_importance()
>>> # or
>>> model_eval.run_feature_importance(
...     desired_fpr=0.01,
...     sample_rate=0.3,
...     random_seed=123456
... )
>>> # or
>>> model_eval.run_feature_importance(
...     desired_fpr=0.01,
...     sample_rate=0.3,
...     random_seed=123456,
...     job_properties={"spark.executor.memory": "5g"},
... )
```

Metric [¶](#)

Bases: Enum

Supported metrics in model evaluations.

ACCURACY class-attribute instance-attribute [↗](#)

```
ACCURACY = auto()
```

ALERT_RATE class-attribute instance-attribute [↗](#)

```
ALERT_RATE = auto()
```

FALSE_DISCOVERY_RATE class-attribute instance-attribute [↗](#)

```
FALSE_DISCOVERY_RATE = auto()
```

FALSE_NEGATIVES class-attribute instance-attribute [↗](#)

```
FALSE_NEGATIVES = auto()
```

FALSE_NEGATIVE_RATE class-attribute instance-attribute [↗](#)

```
FALSE_NEGATIVE_RATE = auto()
```

FALSE_POSITIVES class-attribute instance-attribute [↗](#)

```
FALSE_POSITIVES = auto()
```

FALSE_POSITIVE_RATE class-attribute instance-attribute [↗](#)

```
FALSE_POSITIVE_RATE = auto()
```

F_MEASURE class-attribute instance-attribute [↗](#)

```
F_MEASURE = auto()
```

INFORMEDNESS class-attribute instance-attribute [↗](#)

```
INFORMEDNESS = auto()
```

MARKEDNESS class-attribute instance-attribute [↗](#)

```
MARKEDNESS = auto()
```

NEGATIVE_PREDICTIVE_VALUE class-attribute instance-attribute [↗](#)

```
NEGATIVE_PREDICTIVE_VALUE = auto()
```

POSITIVE_PREDICTIVE_VALUE class-attribute instance-attribute [↗](#)

```
POSITIVE_PREDICTIVE_VALUE = auto()
```

PRECISION class-attribute instance-attribute [↗](#)

```
PRECISION = auto()
```

PREDICTED_NEGATIVES class-attribute instance-attribute [↗](#)

```
PREDICTED_NEGATIVES = auto()
```

PREDICTED_POSITIVES class-attribute instance-attribute [↗](#)

```
PREDICTED_POSITIVES = auto()
```

RECALL class-attribute instance-attribute [↗](#)

```
RECALL = auto()
```

SPECIFICITY class-attribute instance-attribute [↗](#)

```
SPECIFICITY = auto()
```

TRUE_NEGATIVES class-attribute instance-attribute [↗](#)

```
TRUE_NEGATIVES = auto()
```

TRUE_POSITIVES class-attribute instance-attribute [↗](#)

```
TRUE_POSITIVES = auto()
```